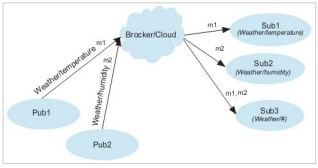


Figure 1: Publish-subscribe example

## QoS levels

MQTT supports three levels of QoS, specified by each published message and while subscribers are connecting.

*QoS 0 (Maximum once):* This is also known as 'fire and forget'; no acknowledgement is sent by the receiver.

*QoS 1 (At least once):* Each published message will be acknowledged using PUBACK; the sender retransmits a message if no acknowledgement is received within a time-out by setting the DUP flag.

*QoS 2 (Exactly once):* Sender and receiver exchange *PUBLISH*, *PUBREC*, *PUBREL*, *PUBCOMP* to ensure assured delivery of messages without duplicates.

Level 0 is faster but less reliable, whereas Level 2 is the most reliable, yet slow.

Published messages will be delivered to subscribers with the 0x3 packet type, and QoS level will be downgraded to the specified value of the subscriber. For example, if the publisher QoS is 2 and the subscriber QoS is 1, the messages will be delivered to the subscriber using QoS 1 only.

***Retain flag:*** If the retain flag part of header flags is set, subscribers can get the last message published on this topic, which is retained by the broker.

***Clean session:*** By default, this flag is set in 'connect flags' by subscribers, and published messages will be stored and delivered when they disconnect and reconnect again. We may disable this flag to discard messages published in between.

***Last will and testament (LWT):*** Clients can specify a will topic, will message or will QoS and set the 'will flag' to notify other clients when they terminate abnormally without the DISCONNECT message.

***Keep alive timer:*** When there is no message flow for a particular duration specified while connecting, the client sends a PINGREQ and the broker replies by sending a PINGRESP indicating the 'liveness' of the connection.

## MQTT brokers

MQTT clients exchange messages via the broker node. The broker is not identical to a typical server, as apart from message reception and delivery, it has little functionality. For additional functionality like logging, message persistence, visualisation, analytics, Web integration, etc, one should consider additional subscribers or develop plugins for the broker.

| Packet Type (4 bits) | Header Flags (4 bits) |
|---|---|
| Remaining Length (No. of byte:min-1, max-4) | |
| Optional header elements (protocol, flags, keep alive etc.) | |
| Payload (client id, will message, username, password etc. | |

Figure 2: MQTT packet header

| Packet Type | Code | Packet Type | Code |
|---|---|---|---|
| Reserved | 0 | SUBSCRIBE | 8 |
| CONNECT | 1 | SUBACK | 9 |
| CONNACK | 2 | UNSUBSCRIBE | 10 |
| PUBLISH | 3 | UNSUBACK | 11 |
| PUBLICK | 4 | PINGREQ | 12 |
| PUBREC | 5 | PINRESP | 13 |
| PUBREL | 6 | DISCONNECT | 14 |
| PUBCOMP | 7 | Reserved | 15 |

Figure 3: MQTT packet types

Mosquitto is an Eclipse IOT project, lightweight broker implementation written in C and it supports MQTT protocol versions 3.1 and 3.1.1. For building Mosquitto, install the dependency *libcares-devel*.

Download Mosquitto-1.4.9.tar.gz (or any recent stable version) from *https://mosquitto.org/download/*, extract it and switch into…

```
tar -zxvf  mosquitto-1.4.9.tar.gz
cd mosquitto-1.4.9
mkdir build && cd build
cmake ..   #install any other missing dependencies
make
make install
```

You can run the broker using the command *mosquitto* located in */usr/local/sbin*, which is copied from *build/src* during installation. It runs on TCP Port 1883, by default.

Other open source implementations of the MQTT broker are:

- Emqttd, Vernemq written in Erlang
- Mosca in Nodejs (npm package)
- Surgemq in Go language
- Moquette, Vertx-mqtt-broker in Java
- HBMQTT in Python

Apache ActiveMQ, ApacheApollo and RabitMQ also support the protocol through their plugins.

A majority of these brokers target the latest version 3.1.1, but some still support v3.1 (IBM version) for backward compatibility.

## MQTT clients

There are many implementations of MQTT clients, but we'll only discuss the most popular. One is the Eclipse Paho client library with support for many languages and the other is from the Mosquitto library.

Paho provides APIs for two types of clients for operations like connect, publish, subscribe, unsubscribe, etc.

1. *Synchronous API:* The above operations are blocked until